

## 110612117 張仲瑜 HW1 Report

### Part 0:

1. 原先提供的 function 利用停用詞過濾的方法，來移除資料中對我們判斷內容、情感無幫助的字，包括介係詞(on、at)、連接詞(and、or)、冠詞(a、and)。利用 nltk 提供的停用詞庫轉換成小寫來一一跟切割成 token 的原資料詞比對，相同就刪掉
2. 我自己寫的 function 先利用了他的停用詞比對處理資料，再經過觀察資料後，刪除資料中混和的一些不必要元素像 html 的換行語法、連續的空格、都換成小寫、特殊字元刪除、數字刪除等，來刪減不重要的元素。

### Part 1:

1. Perplexity 是評估語言模型好壞的指標之一，每個位置出現狗字的機率相乘，也就是預測整個句子各種字不同排列的機率，取 log 後做平均再取負號與 2 的指數，因此 perplexity 越小，對資料序列的預測就越準確。  
  
會影響 Perplexity 的包含語言模型的設計、訓練數據的合理性、詞句的分布、語言特殊結構等等，當訓練資料量越大，分布越平均時，Perplexity 通常會較低，反之則上升。

2.

```
Perplexity of ngram: 1000.0000000000002
feature num = 10000
100%| 40000/40000 [00:08<00:00, 4448.38it/s]
100%| 40000/40000 [01:29<00:00, 449.03it/s]
100%| 10000/10000 [00:21<00:00, 464.33it/s]
F1 score: 0.8376, Precision: 0.8384, Recall: 0.8377
PS C:\Users\vince\OneDrive\桌面\AI Hm2>
```

未進行 preprocess

F1 score: 0.8376

Precision: 0.8384

Recall: 0.8377

```
[nltk_data] C:\Users\vince\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
Perplexity of ngram: 1000.0
feature num = 10000
100%| 40000/40000 [00:06<00:00, 5904.86it/s]
100%| 40000/40000 [01:18<00:00, 507.84it/s]
100%| 10000/10000 [00:23<00:00, 424.81it/s]
F1 score: 0.812, Precision: 0.8138, Recall: 0.8122
PS C:\Users\vince\OneDrive\桌面\AI Hm2>
```

使用 Remove\_stopwords preprocess

F1 score: 0.812

Precision: 0.8138

Recall: 0.8122

```
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\vince\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
Perplexity of ngram: 1000.0
feature num = 10000
100%| 40000/40000 [00:06<00:00, 5801.19it/s]
100%| 40000/40000 [01:22<00:00, 485.70it/s]
100%| 10000/10000 [00:22<00:00, 439.40it/s]
F1 score: 0.8261, Precision: 0.8277, Recall: 0.8263
PS C:\Users\vince\OneDrive\桌面\AI Hm2>
```

使用自訂 preprocess

F1 score: 0.8261

Precision: 0.8277

Recall: 0.8263

## Part 2:

### 1. Bert 的 Pre-training step 包括兩個步驟

第一個是 Mask Language Model(MLM)，將一個句子的 n 隨機字用隨機字替換或用 mask 替換，然後放入 Encoder 中，在將輸出向量做 linear transform、softmax，以得到預測被蓋起來的字為何的分布。預測目標追求與原字接近，用來讓模型理解一個句子裡字跟字之間的關係

第二個是 Next Sequence Prediction(NSP)，會將兩個句子隨機拼湊起來，然後前面加上[CLS]符號、中間加上[SEP]分隔符號後，將 CLS 的輸出做 Linear，得到二元化的分類，判斷兩個句子是否為相接的， 用來學習兩個句子間的邏輯關係

### 2. 以下提及 Bert 的初始化皆為已經 Pre-trained 過的參數

開頭皆有 CLS 符號、連接句子用 SEP 符號、需要標註資料。

A. Input 為 sequence, Output 為 class，像是這次做的

sentiment analyze，一樣透過[CLS]的 encoder 輸出做轉換後做二元判斷達成。

B. Input 為 sequence, Output 與 input 長度一樣，像是詞性標

註，將每個字通過 Encoder 的輸出做轉換得到各詞性機率。

C. Input 為 two sequence, Output 為 class，像是 NLI 判斷兩個句子屬於矛盾還是合理、立場分析等等，一樣透過 [CLS]Encoder 後結果轉換做分類。

D. Input 為文章與問題(sequences), Output 為兩個正整數，兩個正整數字序之間的字即為答案，像是閱讀文章後 QA，透過兩個隨機初始化的向量(Query, key)跟每個 Encoder 輸出做 inner product，做 softmax 後分別選擇兩者機率最高作為頭尾字序答案，答案需在文章內。

3. DistilBert 與 Bert 相比模型大小更小(Bert 的 40%)，速度更快(60%)，訓練層數為 Bert 的一半，減少了 hidden\_vector\_size，Loss function 由三種組成，大致參考 Bert 的預測或與 Bert 相關，就像學生學習老師一樣。優勢為擁有更好的任務移植性，適用於資源受限的場合、降低伺服器的成本。

4. Transformer 的核心為多頭自注意力機制，透過對不同字做不同組線性變換將輸入的每個字分成多頭，用以提供該字在不同種類型上對其他字有高或低的影響，Transformer 模型在 Encoder 和 Decoder 中都使用了多頭注意力機制來對輸入進行特徵提取，且增加了自注意力機制，每個注意力頭都有自己的

權重矩陣，也有多個子空間可以專注在不同目標，在不同位置的語意訊息結合上有很好的效果。Bert 則站在巨人的肩膀上，使用 Transformer Encoder 部分作為基礎架構再進行上方回答過的 Pre-trained，透過無監督式大量學習語法和語意後，完成各種語言處理任務。

5.

```
[ ] !python main.py --model_type BERT --preprocess 0 --part 2
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...  
[nltk_data] Package stopwords is already up-to-date!  
Some weights of the model checkpoint at distilbert-base-uncased were not used when initializing DistilBertModel: ['vocab_projector.weight', 'vocab_tr  
- This IS expected if you are initializing DistilBertModel from the checkpoint of a model trained on another task or with another architecture (e.g.  
- This IS NOT expected if you are initializing DistilBertModel from the checkpoint of a model that you expect to be exactly identical (initializing a  
Epoch: 0  
Epoch: 0, F1 score: 0.9202, Precision: 0.9206, Recall: 0.9202, Loss: 0.2583
```

未進行 preprocess

F1 score: 0.9202

Precision: 0.9206

Recall: 0.2583

```
!python main.py --model_type BERT --preprocess 1 --part 2
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...  
[nltk_data] Package stopwords is already up-to-date!  
Some weights of the model checkpoint at distilbert-base-uncased were not used when initializing DistilBertModel: ['vocab_transform.weight', 'vocab_layer_norm.bia  
- This IS expected if you are initializing DistilBertModel from the checkpoint of a model trained on another task or with another architecture (e.g. initializing  
- This IS NOT expected if you are initializing DistilBertModel from the checkpoint of a model that you expect to be exactly identical (initializing a BertForSequ  
0th Epoch:  
0th Epoch: , F1 score: 0.9209, Precision: 0.9213, Recall: 0.9209, Loss: 0.2766
```

使用 Remove\_stopwords preprocess

F1 score: 0.9209

Precision: 0.9213

Recall: 0.9209

Loss: 0.2766

Part 3:

1. Vanilla RNN 每個時間步的輸出僅受當下的時間步與上一個時間步的影響，在處理長序列時容易出現梯度消失和梯度爆炸；  
LSTM 則透過遺忘門、輸出門、輸入門來維持和處理長期記憶。遺忘門控制是否遺忘先前的狀態，輸入門控制新訊息對狀態的影響，輸出門則決定要把哪些狀態傳下去。
2. nn.Embedding 輸入參數為學習資料的一維向量，在這個作業裡就是 20000 筆資料，輸出為嵌入向量，將輸入轉為低維的連續形向量；nn.LSTM 輸入參數依序為序列長度、訓練批次大小(若 batch\_first = true 則前兩者相反)、輸入資料大小、LSTM 隱藏狀態的大小、LSTM 的層數、有沒有用雙向的 LSTM，輸出則為批次大小\*序列長度\*LSTM 隱藏狀態；如果是雙向 LSTM，nn.linear 輸入為 2\*隱藏狀態大小，輸出則為用來判斷分類的數值向量；nn.dropout 則以輸入的機率隨機丟棄結果，防止過擬合發生。
3. 看到這段文字代表我三天內除不完錯-20 分比較虧 QQ

就不更新了，會附上失敗檔

## Discussion

1. 在了解這些算法後可以發現，限制從最一開始只能透過 ngram 做 n 個 token 的組與下組頻率的統計來預測；到 LSTM 可以有長期的記憶性，但對長距離的依賴關係仍嫌不足；再到可以對學習前後字序不同關注點的特徵，突破的背後除了為解決突破前者不足處的需求，也跟硬體效能的成長有所關聯，才能處理複雜度較高的問題。
2. 了解模型的理論與實作模型的難度不相上下，要在有限的時間內看完所有提供的學習資料相當不容易，尤其是傳遞參數的接口經常因為理解不完全而出錯，因此花費許多時間在 debug 與了解，尤其在 reshape 上令人頭痛，下次需要提前花更大量的時間來了解資料的維度，才能事半功倍。